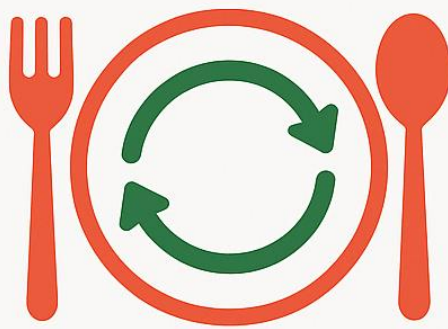


Ciclo Superior de Desarrollo de Aplicaciones Web

Proyecto Final de Grado Superior (TFGS)



FoodSynk

FoodSynk

Gestor web de menús, recetas y lista de la compra

Autora: Nuria

2º DAW — Curso 2025–2026

Enero – Marzo 2026

Introducción	4
Título:	5
Descripción:	5
Objetivo:	5
Enlaces de referencia:	6
Viabilidad:	6
Stack tecnológico e infraestructura:	7
Material extra necesario:	7
Nota final:	8
Justificación	9
Objetivos	10
1. Base funcional (cuentas y seguridad)	10
2. Experiencia de usuario	10
3. Roles y vistas de usuario	11
4. Lista de la compra	11
5. Recetario	11
6. Gestión de menús	12
7. Calidad técnica y mantenimiento	12
Temporalización	14
Fase 1: Análisis y diseño (Semana 1 - Semana 2)	¡Error! Marcador no definido.
Fase 2: Configuración del entorno y estructura base (Semana 3)	¡Error! Marcador no definido.
Fase 3: Módulo de usuarios y seguridad (Semana 4 - Semana 5)	¡Error! Marcador no definido.
Fase 4: Gestión de recetas (Semana 6 - Semana 7)	¡Error! Marcador no definido.
Fase 5: Creación de menús personalizados (Semana 8 - Semana 9)	¡Error! Marcador no definido.
Fase 6: Generación automática de la lista de la compra (Semana 10 - Semana 11)	¡Error! Marcador no definido.
Fase 7: Interfaz y experiencia de usuario (Semana 12)	¡Error! Marcador no definido.
Fase 8: Pruebas, ajustes y despliegue (Semana 13 - Semana 14)	¡Error! Marcador no definido.
Fase 9: Mejoras futuras (Versión 2)	¡Error! Marcador no definido.

Introducción

A la hora de hacer la compra semanal, es muy común olvidar algún ingrediente o calcular mal las cantidades necesarias, especialmente cuando no se ha planificado con claridad qué recetas se van a preparar o cuántas comidas se realizarán. Esto provoca que muchas veces haya que volver al supermercado o improvisar platos con lo que se tiene en casa, lo que genera pérdida de tiempo y desorganización.

FoodSynk surge precisamente para resolver ese problema. Su objetivo es facilitar la planificación de las comidas de toda la semana y automatizar la generación de la lista de la compra, teniendo en cuenta las recetas elegidas, los ingredientes necesarios y el número de personas que van a comer.

De esta manera, el usuario podrá organizar sus menús de forma práctica y visual, evitando olvidos y compras innecesarias. Además, la aplicación pretende fomentar una alimentación más variada y equilibrada, reduciendo el tiempo dedicado a pensar cada día qué cocinar y optimizando el proceso de compra.

Título:

FoodSynk — Gestor web de menús semanales, recetas y lista de la compra.

Descripción:

FoodSynk es una aplicación web diseñada para ayudar a organizar las comidas de forma más sencilla y práctica. Está pensada para personas que no quieren complicarse todos los días pensando qué cocinar. Un caso típico es el de alguien que necesita planear el menú semanal para su familia, con varias comidas al día y para varias personas.

La idea es que el usuario pueda crear menús personalizados por días y por comidas (desayuno, almuerzo, cena) y adaptarlos al número de personas. Para eso cuenta con un recetario consultable donde se pueden buscar recetas, filtrarlas por categorías y leer comentarios de otros usuarios. Además, cada usuario registrado puede guardar sus recetas favoritas, subir las suyas propias, editarlas o eliminarlas, así como comentar en las de otros.

La parte más característica del proyecto es la lista de la compra automática. Una vez creado el menú, la aplicación calcula qué ingredientes se necesitan según el número de platos, días y personas, y genera la lista de la compra. Esta lista se puede editar a mano (por ejemplo, si ya se tiene un ingrediente en casa), y el usuario puede descargarla o imprimirla para usarla directamente al hacer la compra.

La aplicación contempla tres tipos de usuario:

Usuario no registrado: puede ver el recetario, filtrar recetas y leer comentarios, pero no puede crear ni guardar nada.

Usuario registrado: tiene acceso a todas las funciones disponibles actualmente: crear, editar y eliminar sus recetas, gestionarlas y descargar su lista de la compra.

Administrador: tiene permisos para eliminar recetas, comentarios o usuarios que incumplan las normas. Este rol está planificado para una fase posterior del desarrollo.

FoodSynk está diseñada para ser escalable, de forma que en el futuro se puedan añadir más funciones, como el cálculo de los valores nutricionales de cada plato o un sistema de recomendaciones según los gustos del usuario.

Objetivo:

El objetivo principal de FoodSynk es ofrecer una herramienta útil y práctica que ayude a las personas a organizar su alimentación de forma sencilla, evitando tener que pensar cada día qué cocinar y facilitando el proceso de compra mediante listas generadas automáticamente. A nivel personal, este proyecto me permite aplicar de forma real los conocimientos adquiridos durante el curso y utilizarlo como parte de mi portafolio profesional, demostrando mis capacidades en diseño, desarrollo y organización de un proyecto completo.

Enlaces de referencia:

1. [Plan to Eat](#): Permite importar recetas, organizarlas, planificar las comidas y generar automáticamente la lista de la compra según lo que hayas puesto en tu calendario. Es de pago, pero es una aplicación muy interesante por la interfaz simple que muestra.
2. [Mealime](#): Es una app que simplifica la planificación de comidas saludables, con recetas de rápido preparo y listas de compra vinculadas. Esta app me ha gustado por los filtros (dieta, tiempo...) y el diseño UX.
3. [Eat This Much](#): Genera planes de comidas automáticos según preferencias, presupuesto, dieta, etc., y crea la lista de compras. Es más “avanzada”: ya empieza a mezclar nutrición con planificación.

Existen muchas más aplicaciones similares, pero la revisión de estas tres ha sido suficiente para tener una referencia clara de qué funcionalidades incluir y cómo orientar el diseño de la interfaz.

Viabilidad:

El proyecto es totalmente viable. Utiliza tecnologías modernas y bien documentadas como React, Laravel y MySQL, que cuentan con una comunidad amplia y recursos de aprendizaje accesibles. No requiere infraestructura compleja: se puede comenzar con una base de datos con recetas de ejemplo e ir ampliándola progresivamente. Además, al estar planteado de forma modular, es posible desarrollar primero las funcionalidades principales (recetas, menús y lista de la compra) y añadir mejoras en fases posteriores sin necesidad de rehacer el sistema desde cero.

Stack tecnológico e infraestructura:

- Frontend:
 - **React** se utilizará para desarrollar la interfaz de usuario, ya que actualmente es una de las librerías más utilizadas en el desarrollo web. Además, me gustaría aprender a trabajar con ella porque considero que es una herramienta muy potente y con gran demanda en el mercado laboral.
 - Se combinará con **Tailwind CSS** para la parte visual. Aunque ya tengo experiencia con Bootstrap, prefiero los acabados que ofrece Tailwind y la flexibilidad que proporciona para adaptar los diseños. Creo que ambos elementos, React y Tailwind, pueden combinar muy bien para obtener una interfaz moderna, funcional y atractiva.
- Backend:
 - **Laravel** (PHP) es el framework elegido para el backend. Es una herramienta completa, bien documentada y con una curva de aprendizaje accesible. Para la autenticación se utiliza **Laravel Breeze** instalado en modo API, junto con **Laravel Sanctum** para la gestión de tokens de autenticación. Esta configuración es la adecuada para un proyecto con frontend separado en React. Además, Laravel es ampliamente usado en el ámbito profesional, lo que hace que sea una elección útil tanto para el proyecto como para mi formación.
- Base de datos:
 - Se usará **MySQL** como sistema de base de datos, por su estabilidad, compatibilidad con Laravel y facilidad de configuración.
 - Para la conexión entre la base de datos y el backend se usará **Eloquent**, el ORM que viene integrado en Laravel, ya que ambos funcionan de forma muy fluida y simplifican mucho el trabajo con las tablas y las relaciones.
- Despliegue:
 - La aplicación se desplegará en un VPS con Nginx, con copias de seguridad automatizadas.
 - Se utilizará GitHub para control de versiones y GitHub Actions para automatizar los despliegues (CI/CD).
 - Como alternativa, se podrá alojar el frontend en Vercel y el backend en un servidor VPS o servicio como Railway.

Material extra necesario:

- Dominio propio con certificado SSL (Let's Encrypt).
- Correo SMTP para notificaciones y recuperación de contraseñas.
- Almacenamiento de imágenes en el servidor.

- Iconos y gráficos de Flaticon, Heroicons o Tabler Icons.
- Tipografías de Google Fonts, como Inter o Poppins.
- Logo y favicon creados con Canva o Figma.

Nota final:

Aunque este es el stack tecnológico que prefiero utilizar, es posible que algunos elementos cambien dependiendo de las necesidades técnicas del proyecto o de los requisitos específicos establecidos en el protocolo de evaluación. La idea es mantener la flexibilidad para adaptar la aplicación a las circunstancias y asegurar que sea viable y funcional en cualquier entorno de desarrollo.

Justificación

La idea de este proyecto surge de una necesidad cotidiana muy común: la dificultad de planificar las comidas semanales de forma eficiente y práctica. Aunque existen aplicaciones similares, muchas resultan confusas, poco intuitivas o están pensadas únicamente para usuarios con conocimientos avanzados de cocina. Mi intención con este proyecto es crear una herramienta sencilla, clara y visual, que cualquier persona pueda utilizar sin complicaciones.

Lo que diferencia a FoodSynk de otras propuestas es su enfoque centrado en la comodidad del usuario y la personalización de la experiencia. La idea es que cada persona pueda adaptar la aplicación a su estilo de vida, filtrando recetas según sus preferencias: por tiempo de preparación, dificultad o tipo de plato. De este modo, no solo se facilita la planificación semanal, sino que también se anima a las personas a variar su alimentación sin tener que dedicar tanto tiempo a decidir qué cocinar.

Además, la generación automática de la lista de la compra, ajustada al número de comidas y personas, aporta un valor añadido muy práctico. Evita los olvidos frecuentes al hacer la compra y ahorra tiempo, reduciendo la necesidad de volver al supermercado por haber olvidado algo. En conjunto, el proyecto busca equilibrar simplicidad, utilidad y personalización, convirtiéndose en una herramienta realmente útil para la organización diaria.

A nivel técnico y personal, elegí este proyecto porque reúne varios elementos que me permiten aplicar de forma práctica los conocimientos adquiridos durante el curso. Se trata de un desarrollo que abarca múltiples operaciones CRUD —para la gestión de recetas, menús y listas de la compra—, lo que lo convierte en un proyecto completo desde el punto de vista del backend y del diseño de base de datos, pero sin perder la sencillez ni la claridad en su estructura.

Además, su carácter escalable lo hace especialmente interesante, ya que podría evolucionar fácilmente con nuevas funcionalidades, como el cálculo automático de calorías o valores nutricionales orientados a distintos perfiles de usuario. Considero que es un proyecto equilibrado: suficientemente amplio para demostrar un dominio sólido de la programación web, pero lo bastante accesible como para desarrollarlo de forma eficiente con una buena organización.

Objetivos

Objetivo general:

Desarrollar una aplicación web práctica y sencilla que permita al usuario planificar sus menús semanales, consultar recetas y generar automáticamente una lista de la compra adaptada a sus necesidades, facilitando así la organización de las comidas y el ahorro de tiempo en el día a día.

Objetivos específicos:

A continuación, se detallan los objetivos específicos del proyecto, organizados por categorías.

Para su desarrollo se establecen dos niveles de prioridad:

- **MVP (Minimum Viable Product):** funcionalidades esenciales que deben estar implementadas para que la aplicación sea completamente funcional.
- **V2 (Versión 2):** funcionalidades adicionales o ampliaciones que pueden incorporarse en una segunda fase, una vez finalizada la versión principal.

1. Base funcional (cuentas y seguridad)

- **[MVP]** Permitir el registro de nuevos usuarios.
 - **[MVP]** Implementar el inicio y cierre de sesión (login/logout) de forma segura.
 - **[MVP]** Permitir la edición de los datos personales del usuario.
 - **[MVP]** Ofrecer la posibilidad de eliminar la cuenta de usuario.
 - **[MVP]** Implementar un sistema para restablecer la contraseña mediante correo electrónico.
 - **[V2]** Añadir verificación de correo electrónico tras el registro.
-

2. Experiencia de usuario

- **[MVP]** Diseñar una interfaz simple, clara e intuitiva que facilite la navegación por las distintas secciones.
- **[MVP]** Incorporar mensajes visuales de confirmación o error al realizar acciones (guardar, eliminar, comentar, etc.).

- **[V2]** Mejorar la accesibilidad general de la aplicación (contraste, etiquetas ARIA, tamaño de texto, etc.).
-

3. Roles y vistas de usuario

- **[MVP]** Crear una vista para el usuario no registrado, que permita consultar y filtrar recetas sin poder modificarlas.
 - **[MVP]** Desarrollar una vista para el usuario registrado, que le permita crear, guardar, comentar recetas, generar menús y listas de la compra.
 - **[MVP]** Incluir un rol de administrador con acceso para gestionar usuarios, recetas y comentarios.
 - **[V2]** Desarrollar un panel visual unificado para el administrador que centralice la gestión de la aplicación.
-

4. Lista de la compra

- **[MVP]** Calcular automáticamente los ingredientes necesarios a partir de las recetas incluidas en el menú semanal.
 - **[MVP]** Adaptar las cantidades en función del número de personas y comidas seleccionadas.
 - **[MVP]** Permitir eliminar uno o varios ingredientes de la lista en caso de que el usuario ya los tenga en casa.
 - **[MVP]** Incluir la opción de imprimir o descargar la lista de la compra.
 - **[V2]** Organizar los ingredientes por categorías (verduras, carnes, lácteos, etc.) para facilitar la compra.
-

5. Recetario

- **[MVP]** Mostrar todas las recetas disponibles.
- **[MVP]** Permitir filtrar recetas por categoría, tiempo de preparación o dificultad.

- **[MVP]** Permitir crear, editar y eliminar recetas propias.
 - **[MVP]** Añadir sistema de comentarios en las recetas.
 - **[MVP]** Permitir guardar recetas y marcarlas como favoritas.
 - **[V2]** Añadir etiquetas personalizadas (rápidas, saludables, económicas, etc.).
-

6. Gestión de menús

- **[MVP]** Incluir un formulario inicial que solicite datos para la planificación del menú:
 - Número de días.
 - Número de personas.
 - Número de comidas diarias.
 - Platos por comida.
 - Si desea repetir alguna receta.
 - **[MVP]** Generar automáticamente una hoja de menú editable según las respuestas del formulario.
 - **[MVP]** Desarrollar un asistente guiado que permita al usuario elegir recetas paso a paso (por comidas o platos).
 - **[MVP]** Permitir guardar y reutilizar menús ya creados.
 - **[V2]** Incorporar un sistema de sugerencias personalizadas según los menús o recetas más utilizadas.
-

7. Calidad técnica y mantenimiento

- **[MVP]** Mantener una estructura clara del proyecto separando frontend, backend y base de datos.
- **[MVP]** Validar los datos tanto en el cliente como en el servidor para evitar errores y mejorar la seguridad.

- **[MVP]** Crear un conjunto inicial de recetas de ejemplo (semillas de datos) para realizar pruebas.
- **[V2]** Implementar tests básicos para verificar el correcto funcionamiento de los cálculos y las principales funcionalidades.
- **[V2]** Configurar un entorno de despliegue sencillo con control de versiones y actualizaciones automatizadas.

Temporalización

La planificación original del proyecto se organizó en fases genéricas de desarrollo. Sin embargo, a medida que el proyecto ha avanzado ha sido posible tener un control más preciso del tiempo disponible y de las tareas concretas que quedan por completar. Por eso, la temporalización que se presenta a continuación es una versión actualizada y real, organizada semana a semana desde el punto actual del desarrollo (marzo 2026) hasta la presentación final prevista para el 22 de mayo de 2026. El trabajo se lleva a cabo en tardes y fines de semana, con una dedicación aproximada de 8 semanas efectivas.

Semana 1 (16–22 mar) — Backend: módulo de menús

- Migraciones de las tablas Menu y MenuRecipe, modelo y relaciones.
- MenuController con Laravel Orion y rutas protegidas por autenticación.
- MenuPolicy para prevenir acceso IDOR entre usuarios (OWASP A01) y pruebas en Postman.

Semana 2 (23–29 mar) — Frontend: gestión de menús

- Página de gestión de menús en React: crear, editar y eliminar.
- Conexión frontend ↔ backend del módulo de menús con token real.
- Registro de logs en acciones sensibles (store, update, delete).

Semana 3 (30 mar–5 abr) — Backend: lista de la compra

- Servicio ShoppingListGenerator con lógica menú → receta → ingrediente.
- Endpoints GET y POST /shopping-list con validación de límites anti-DoS (OWASP A04).
- Pruebas completas del módulo en Postman.

Semana 4 (6–12 abr) — Frontend lista de la compra + roles de administrador

- Página de lista de la compra en el frontend.
- Backend: sistema de roles (columna role + middleware CheckRole) y rutas de administración.
- Protección frente a mass-assignment en la actualización de usuarios (OWASP A01).

⚠ Objetivo: todo el desarrollo core debe estar operativo antes de Semana Santa.

Semana 5 (13–20 abr) — Semana Santa

Semana de descanso. No se planifica trabajo de desarrollo.

Semana 6 (21–27 abr) — Tests y documentación técnica

- Tests automatizados con PHPUnit: throttle de autenticación y políticas de acceso (error 403 entre usuarios).
- Verificación manual de inyección SQL en Postman.
- Inicio de la documentación técnica: estructura del sistema, base de datos y endpoints.

Semana 7 (28 abr–4 may) — Pulido de interfaz y documentación

- Revisión de la interfaz: consistencia visual, diseño responsive y gestión de estados de error.
- Documentación de decisiones de diseño, medidas OWASP aplicadas y cierre del diario de desarrollo.
- Preparación del entorno de demo o despliegue en servidor (si aplica).

Semana 8 (5–11 may) — Cierre de documentación

- Cierre y revisión final de toda la documentación del proyecto.
- Limpieza del repositorio: commits ordenados y README actualizado.

⚠ Fecha límite de entrega de documentación: 15 de mayo.

Semana 9 (12–22 may) — Preparación de la presentación

- Solo ajustes menores. No se incorporan nuevas funcionalidades.
- Preparación de la demo funcional para la defensa del proyecto.
- Ensayo de la presentación oral.

⚠ Presentación del proyecto: 22 de mayo de 2026.

Hitos clave

12 de abril: Todo el código core terminado (antes de Semana Santa).
21 de abril: Vuelta al trabajo. Inicio de la fase de tests y documentación.
15 de mayo: Entrega de la documentación clave (fecha límite).
22 de mayo: Presentación del proyecto ante el tribunal.

Definición de la Base de datos:

Para definir la base de datos seguiremos los siguientes pasos:

1. Análisis:
 - a. Descripción de la app y sus actividades
 - b. Descripción de cada elemento de la base de datos: entidades, relaciones y atributos. (descripción similar a los enunciados de clase para los esquemas E/R)
 - c. Realizar el esquema E/R, representando las entidades, relaciones, atributos y cardinalidades, en formato draw.io y png/jpeg
2. Diseño:
 - a. Aplicar las reglas de transformación y obtener las tablas del esquema relacional
 - b. Estudiar de forma razonada la normalización y obtener las nuevas tablas normalizadas
 - c. Estudiar la integridad referencial para cada clave foránea
 - d. Implementar el diagrama del modelo relacional completo en MySQL WorkBench
3. Desarrollo:
 - a. Crear el script de la base de datos y las tablas ajustando los tipos de datos e incluyendo las claves primarias, foráneas y alternativas en cada caso. Añade las restricciones más adecuadas según cada campo de la tabla (Not null, default, check,...) y adecuándose al caso real planteado.
 - b. Crear otro script de datos para añadir al menos 4 registros de cada tabla para realizar las pruebas necesarias de comprobación de las restricciones.
4. Política de acceso:
 - a. Política de usuarios (y roles).
 - b. Asignación de privilegios.
 - c. Gestión de cuentas de usuario, credenciales y control de accesos
 - d. Mejora futura: Automatización de la asignación, revisión y gestión de roles
5. Automatización:
 - a. Al menos un procedimiento almacenado que haga uso de cursores y del manejo de errores mediante handler.
 - b. Funciones
 - c. Triggers.

1. Análisis:

a. Descripción de la aplicación y sus actividades

FoodSynk es una aplicación web orientada a la gestión y organización de recetas culinarias por parte de los usuarios. La plataforma permite crear, almacenar y clasificar recetas mediante ingredientes y categorías, facilitando su consulta y organización dentro de la aplicación. Cada usuario dispone de un espacio donde puede gestionar sus propias recetas, asociarlas con ingredientes y categorías, y decidir si mantenerlas en estado de borrador o publicarlas.

La estructura de la aplicación está diseñada para facilitar la organización de la información culinaria y permitir la planificación de comidas de forma estructurada. La base de datos debe permitir almacenar de forma estructurada toda la información necesaria para el funcionamiento del sistema, garantizando la integridad de los datos y facilitando las relaciones entre los distintos elementos de la aplicación.

- Gestión de usuarios
 - Registro de nuevos usuarios.
 - Autenticación y acceso a la aplicación.
 - Asociación de las recetas con su usuario creador.
- Gestión de recetas
 - Creación de recetas por parte de los usuarios.
 - Edición y eliminación de recetas.
 - Asociación de recetas con ingredientes y categorías.
 - Gestión del estado de las recetas (borrador o publicada).
- Clasificación de información culinaria
 - Registro de ingredientes en la base de datos.
 - Organización de los ingredientes mediante categorías.
 - Relación entre recetas e ingredientes.
- Control y registro de actividad
 - Registro de accesos de los usuarios.
 - Control de acciones relevantes dentro del sistema.
- Planificación y organización de comidas
 - Creación de menús utilizando recetas guardadas.
 - Generación de listas de la compra a partir de los ingredientes de las recetas seleccionadas.
- Interacción entre usuarios
 - Posibilidad de comentar recetas.
 - Valoración o indicación de recetas favoritas.

b. Descripción de cada elemento de la base de datos: entidades, relaciones y atributos

Para el correcto funcionamiento de la aplicación es necesario definir una serie de entidades que representen los distintos elementos que intervienen

en el sistema. Estas entidades permiten estructurar la información que será almacenada en la base de datos y establecer relaciones entre los diferentes datos gestionados por la aplicación. A continuación, se describen las principales entidades que forman parte del modelo de datos del sistema.

- **Usuario**

La entidad Usuario representa a cada persona registrada en la aplicación. Los usuarios pueden acceder al sistema mediante un proceso de autenticación y disponen de un espacio personal desde el que pueden crear y gestionar sus propias recetas. Cada usuario puede crear múltiples recetas, interactuar con recetas de otros usuarios mediante comentarios o valoraciones, y organizar recetas dentro de menús. Los atributos principales incluyen el identificador del usuario, su nombre, dirección de correo electrónico, contraseña y fecha de creación de la cuenta.

- **Receta**

La entidad Receta representa cada uno de los platos registrados en la aplicación. Cada receta pertenece a un usuario, que es el encargado de crearla y gestionarla. Incluye el nombre del plato, las instrucciones, el tiempo estimado de elaboración y una posible imagen asociada. Además, cada receta puede encontrarse en diferentes estados: borrador, mientras se está preparando, o publicada, cuando está finalizada.

- **Ingrediente**

La entidad Ingrediente representa los distintos ingredientes que pueden formar parte de las recetas. Cada ingrediente puede ser utilizado en múltiples recetas, estableciendo una relación de muchos a muchos. Para facilitar su organización, cada ingrediente pertenece a una categoría que permite agruparlos según su tipo.

- **Categoría**

La entidad Categoría permite clasificar los ingredientes en diferentes grupos, facilitando la organización y búsqueda dentro del sistema. Cada categoría puede contener múltiples ingredientes, mientras que cada ingrediente pertenece a una única categoría.

- **Menú**

La entidad Menú permite a los usuarios organizar recetas dentro de una planificación de comidas. Un menú puede estar compuesto por varias recetas seleccionadas por el usuario, permitiendo estructurar las comidas de un periodo determinado.

- **Comentario**

La entidad Comentario representa las opiniones que los usuarios pueden realizar sobre las recetas publicadas. Cada comentario está asociado tanto a un usuario como a una receta.

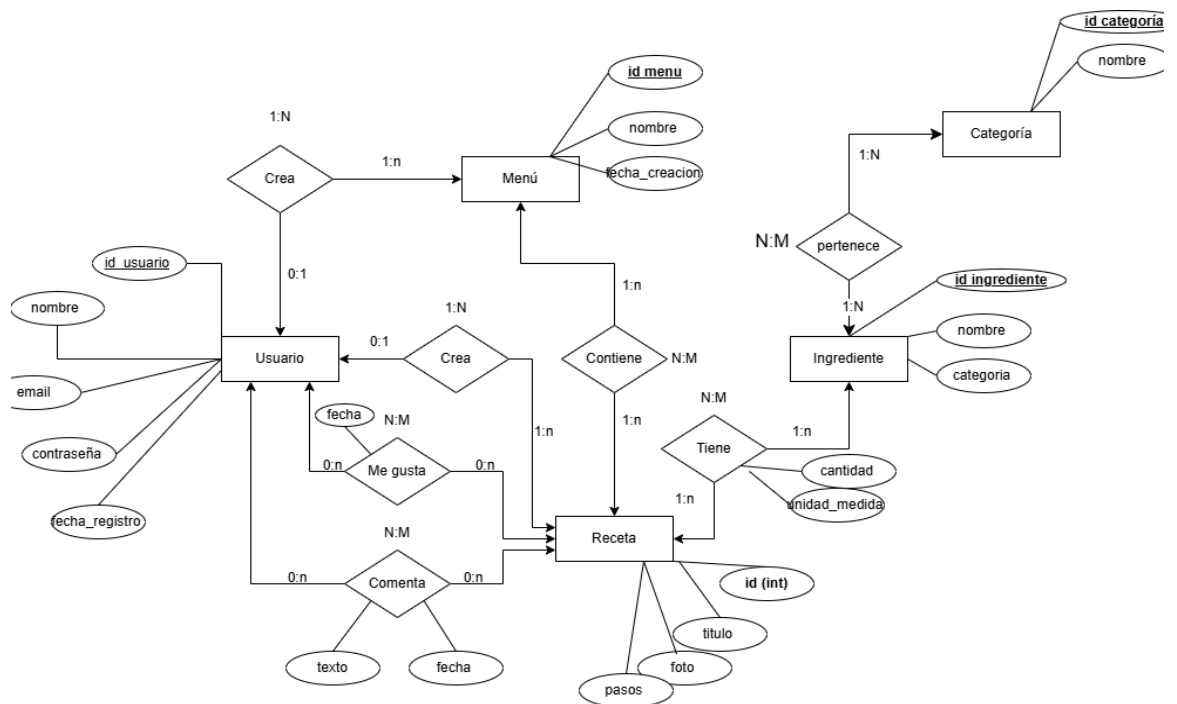
- **Me gusta**

La entidad Me gusta permite registrar las recetas que los usuarios marcan como favoritas. Establece una relación entre usuarios y recetas, indicando qué recetas han sido valoradas positivamente por cada usuario.

- **Registro de accesos**

La entidad Registro de accesos almacena información relacionada con eventos de autenticación dentro de la aplicación, como inicios de sesión, cierres de sesión o registros de nuevos usuarios. Permite mantener un control básico del uso del sistema y facilita tareas de auditoría o seguridad.

c. Realizar el esquema E/R, representando las entidades, relaciones, atributos y cardinalidades, en formato draw.io y png/jpeg



2. Diseño.

a. Aplicar las reglas de transformación y obtener las tablas del esquema relacional

Usuario(ID_Usuario(PK), nombre, email(UQ), contraseña, fecha_registro)

Receta(ID_Receta(PK), titulo, foto, pasos, tiempo_estimado, estado, ID_Usuario(FK))

Ingrediente(ID_Ingrediente(PK), nombre, ID_Categoría(FK))

Categoría(ID_Categoría(PK), nombre)

Menú(ID_Menu(PK), nombre, fecha_creación, ID_Usuario(FK))

Receta_Ingrediente(cantidad, unidad_medida, ID_Receta(FK)(PK), ID_Ingrediente(FK)(PK))

Menu_Receta(ID_Menu(FK)(PK), ID_Receta(FK)(PK))

Me_gusta(fecha, ID_Usuario(FK)(PK), ID_Receta(FK)(PK))

Comentario(texto, fecha, ID_Usuario(FK), ID_Receta(FK))

Registro_Accesos(ID_Registro(PK), tipo_evento, fecha_evento, ID_Usuario(FK))

Notas sobre el esquema relacional:

- Se añade el atributo estado en Receta para soportar los modos borrador y publicada.
- Ingrediente se relaciona directamente con Categoría mediante ID_Categoría(FK), simplificando el modelo anterior.
- Se incorpora Registro_Accesos para reflejar la tabla de auditoría implementada en el sistema.

b. Estudiar de forma razonada la normalización y obtener las nuevas tablas normalizadas

Una vez obtenido el esquema relacional a partir del modelo entidad-relación, es necesario analizar la estructura de las tablas para comprobar que cumplen las reglas de normalización. La normalización permite organizar la información de forma eficiente, evitando redundancias innecesarias y garantizando la consistencia de los datos almacenados. El proceso se ha realizado teniendo en cuenta las tres primeras formas normales (1FN, 2FN y 3FN).

Primera forma normal (1FN). Una tabla cumple la 1FN cuando todos sus atributos contienen valores atómicos: no se almacenan grupos repetidos ni listas dentro de un mismo campo. En este modelo todas las tablas cumplen esta condición, ya que cada atributo almacena un único valor y las relaciones de muchos a muchos se han resuelto mediante tablas intermedias como Receta_Ingrediente, Menu_Receta o Me_gusta.

Segunda forma normal (2FN). Una tabla cumple la 2FN cuando todos los atributos que no forman parte de la clave primaria dependen completamente de ella. Las tablas principales (Usuario, Receta, Ingrediente, Categoría,

Menú) tienen claves primarias simples y todos sus atributos dependen directamente de dicha clave. En las tablas intermedias con clave compuesta (Receta_Ingrediente, Menu_Receta), los atributos adicionales dependen de la combinación completa de la clave primaria.

Tercera forma normal (3FN). Una tabla cumple la 3FN cuando no existen dependencias transitivas entre atributos. En el modelo diseñado, los datos se encuentran separados en entidades distintas para evitar este tipo de dependencias. Por ejemplo, la información de los ingredientes se almacena en la tabla Ingrediente, mientras que su clasificación se gestiona mediante la tabla Categoría, evitando duplicidades.

Tras analizar la estructura del modelo relacional, se concluye que las tablas definidas cumplen las tres primeras formas normales. Esto garantiza que la base de datos evita redundancias innecesarias, mantiene la integridad de los datos y permite gestionar de forma eficiente las relaciones entre los distintos elementos de la aplicación. El modelo obtenido proporciona una base sólida para su implementación en el sistema gestor.

Arquitectura del sistema

FoodSynk sigue una arquitectura de aplicación web separada en dos capas independientes: un backend API REST desarrollado con Laravel y un frontend desarrollado con React. Ambas partes se comunican exclusivamente a través de peticiones HTTP con autenticación por Bearer Token.

Este modelo, conocido como API REST + SPA (Single Page Application), permite que el backend se ocupe únicamente de la lógica de negocio y la gestión de datos, mientras que el frontend se encarga de la presentación y la experiencia de usuario. La separación hace que cada parte sea más fácil de mantener y escalar de forma independiente.

Frontend

El frontend está desarrollado con React y Vite. React construye la interfaz mediante componentes reutilizables, y Vite actúa como el entorno de desarrollo con compilación rápida y recarga en caliente. El enrutamiento entre páginas se gestiona con React Router DOM y los estilos con Tailwind CSS.

Las peticiones HTTP al backend se realizan con Axios, configurado con un interceptor que añade automáticamente el Bearer Token a todas las peticiones. El token se almacena en sessionStorage en lugar de localStorage, lo que reduce el riesgo de exposición en entornos compartidos.

Backend

El backend es una API REST construida con Laravel 11 y PHP. La autenticación se gestiona con Laravel Sanctum mediante Bearer Token. Para la generación automática de endpoints CRUD se utiliza Laravel Orion, que crea las rutas, paginación y filtros a partir del modelo. La validación de datos de entrada se centraliza en clases FormRequest (StoreRecipeRequest, UpdateRecipeRequest, RecipeRequest), siguiendo las recomendaciones OWASP.

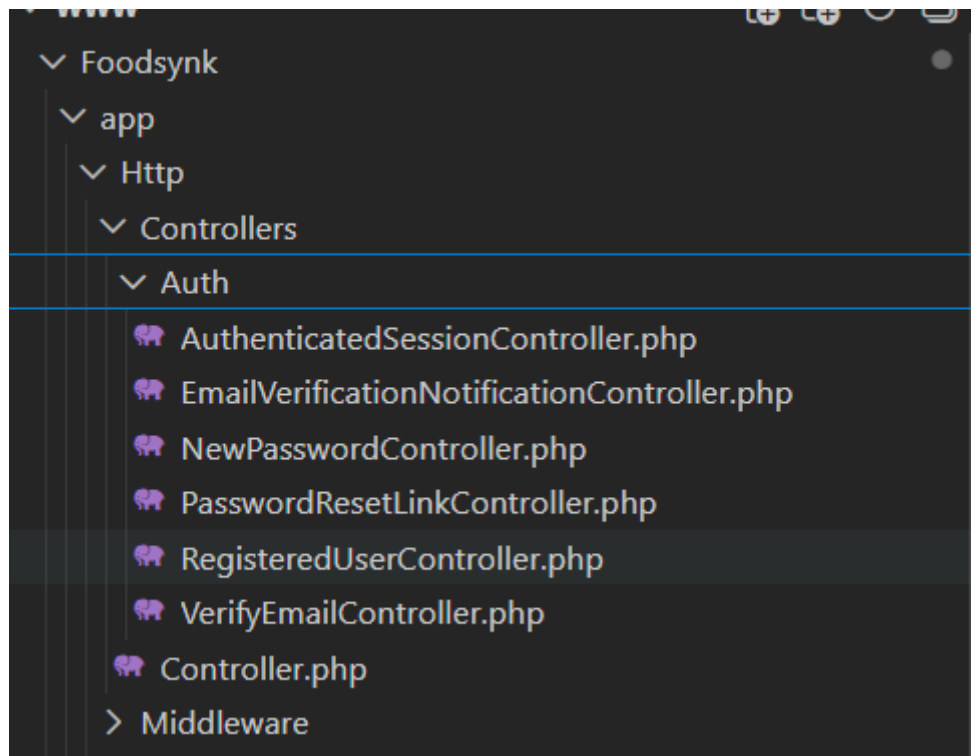


Imagen 5. Archivos de autenticación generados por Breeze en el proyecto backend.

Base de datos

La base de datos es MySQL, gestionada a través de Eloquent ORM. El esquema incluye tablas para usuarios, recetas, ingredientes, categorías, menús, sus relaciones y una tabla de auditoría (user_access_logs) para registrar los accesos al sistema.

Infraestructura y entorno de desarrollo

El entorno local se gestiona con Laragon (Apache, PHP y MySQL en Windows). El control de versiones se realiza con Git y el repositorio se aloja en GitHub. Para las pruebas de la API se utiliza Postman.

```

C:\laragon\www\Foodsynk(main -> origin)
λ php artisan route:list

GET|HEAD / ..... Api\AuthTokenController@login
POST api/login ..... Api\AuthTokenController@login
POST api/logout ..... Api\AuthTokenController@logout
GET|HEAD api/me .....
GET|HEAD api/recipes ..... recipes.index > Api\RecipeController@index
POST api/recipes ..... recipes.store > Api\RecipeController@store
POST api/recipes/batch ..... recipes.batchStore > Api\RecipeController@batchStore
PATCH api/recipes/batch ..... recipes.batchUpdate > Api\RecipeController@batchUpdate
DELETE api/recipes/batch ..... recipes.batchDestroy > Api\RecipeController@batchDestroy
POST api/recipes/search ..... recipes.search > Api\RecipeController@search
GET|HEAD api/recipes/{recipe} ..... recipes.show > Api\RecipeController@show
PUT|PATCH api/recipes/{recipe} ..... recipes.update > Api\RecipeController@update
DELETE api/recipes/{recipe} ..... recipes.destroy > Api\RecipeController@destroy
POST email/verification-notification verification.send > Auth\EmailVerificationNotificationController@store
POST forgot-password ..... password.email > Auth\PasswordResetLinkController@store
POST login ..... login > Auth\AuthenticatedSessionController@store
POST logout ..... logout > Auth\AuthenticatedSessionController@destroy
POST register ..... register > Auth\RegisteredUserController@store
POST reset-password ..... password.store > Auth\NewPasswordController@store
GET|HEAD sanctum/csrf-cookie ..... sanctum.csrf-cookie > Laravel\Sanctum > CsrfCookieController@show
GET|HEAD storage/{path} ..... storage.local
GET|HEAD up .....
GET|HEAD verify-email/{id}/{hash} ..... verification.verify > Auth\VerifyEmailController

Showing [23] routes

```

Imagen 6. Lista de endpoints generados por Laravel Orion (php artisan route:list).

Funcionamiento de la aplicación

En este apartado se describe el flujo de uso de FoodSynk desde el punto de vista del usuario.

Registro e inicio de sesión

Un usuario nuevo puede registrarse proporcionando nombre, correo electrónico y contraseña. El backend valida los datos, crea el usuario y devuelve un token de sesión. Al iniciar sesión, el servidor genera un token que se incluye automáticamente en todas las peticiones siguientes. Al cerrar sesión, el token se elimina del servidor y del navegador.

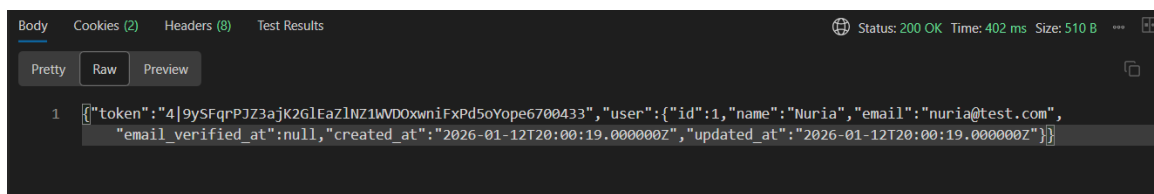


Imagen 7. Token de autenticación devuelto por la API tras un login correcto.

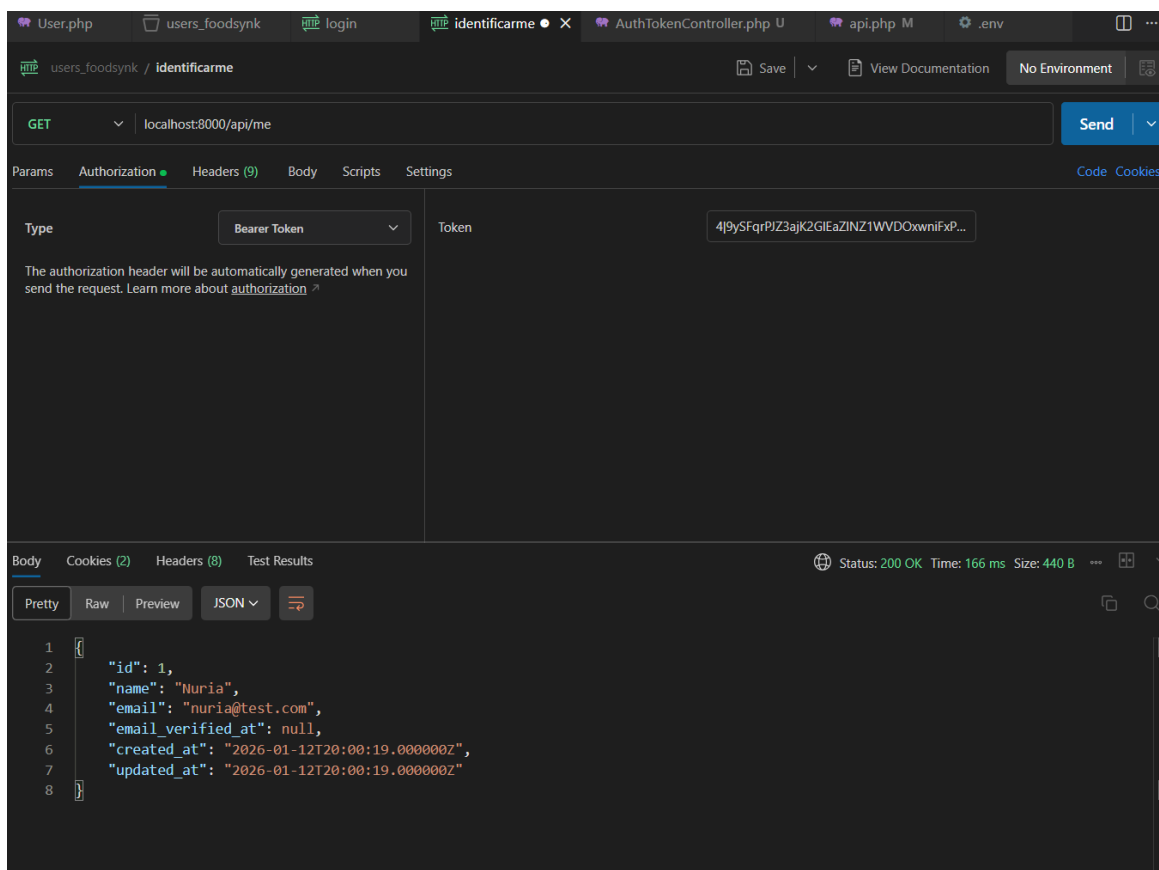


Imagen 8. El endpoint /me devuelve los datos del usuario autenticado con Bearer Token.

Gestión del recetario

Una vez autenticado, el usuario tiene acceso al recetario completo. Puede ver todas las recetas disponibles con sus ingredientes, categorías y tiempo de preparación. Cada usuario registrado puede crear sus propias recetas, editarlas y eliminarlas. El control de acceso garantiza que solo el propietario puede modificar o eliminar sus recetas: si otro usuario lo intenta, el sistema devuelve un error 403 Forbidden.

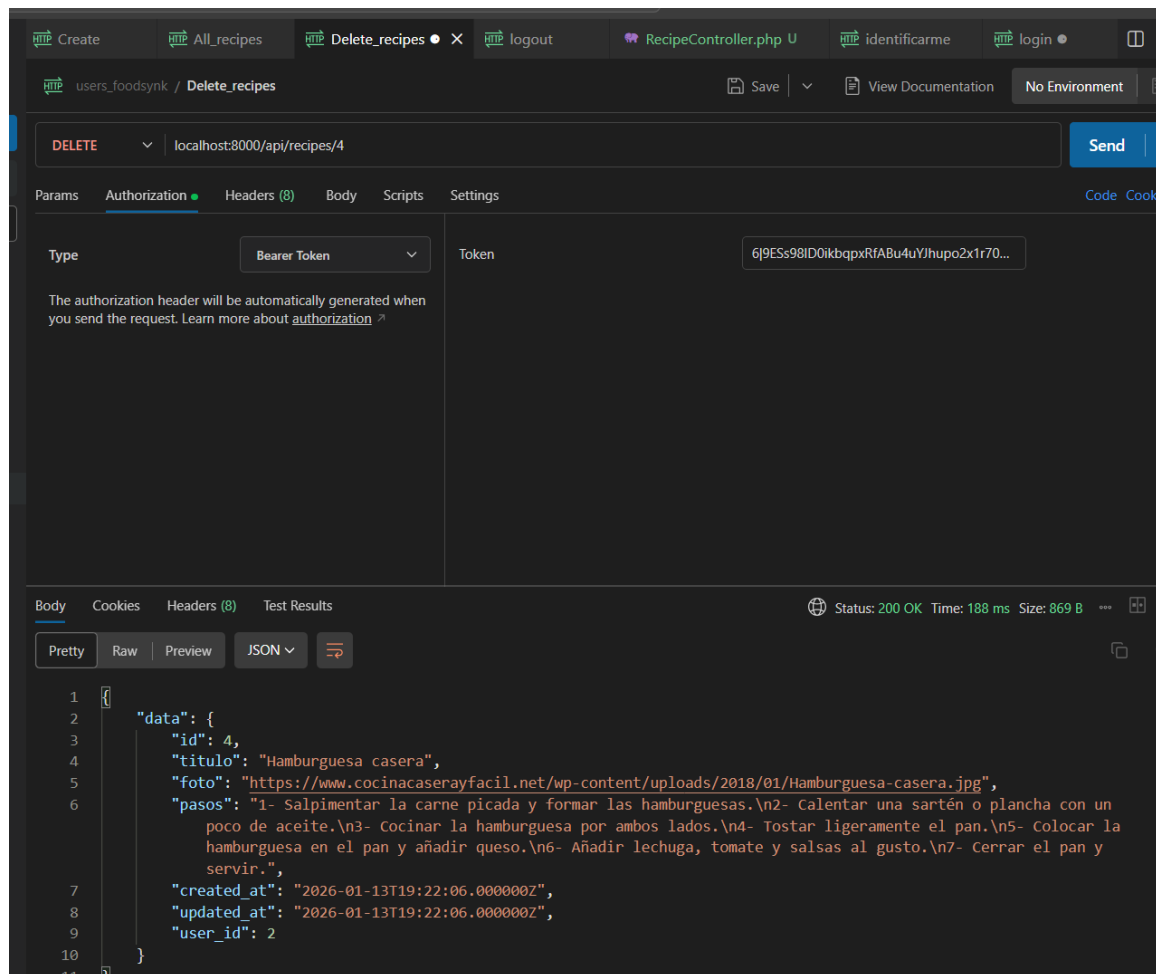


Imagen 9. Intento de eliminar una receta ajena — el sistema devuelve 403 Forbidden.

Exploración y filtros

Las recetas están asociadas a categorías (tipo de alimento, tipo de dieta, alergias) que facilitan la búsqueda y el filtrado según las preferencias del usuario.

Gestión de menús (en desarrollo)

La funcionalidad de creación de menús semanales está planificada para una fase posterior. Permitirá seleccionar recetas para cada día indicando el número de personas, y generará automáticamente la lista de la compra con ingredientes ajustados.

Proceso de desarrollo del proyecto

En este apartado se describe cómo se desarrolló el proyecto FoodSynk desde el inicio, basándose en las entradas del diario de desarrollo.

Planificación inicial y diseño del sistema

El proyecto comenzó el 10 de enero de 2026 con la instalación del entorno de desarrollo con Laragon. En paralelo se diseñó la base de datos, tomando una decisión importante: las categorías de los ingredientes pasaron de ser un atributo a convertirse en una entidad propia, mejorando la escalabilidad.

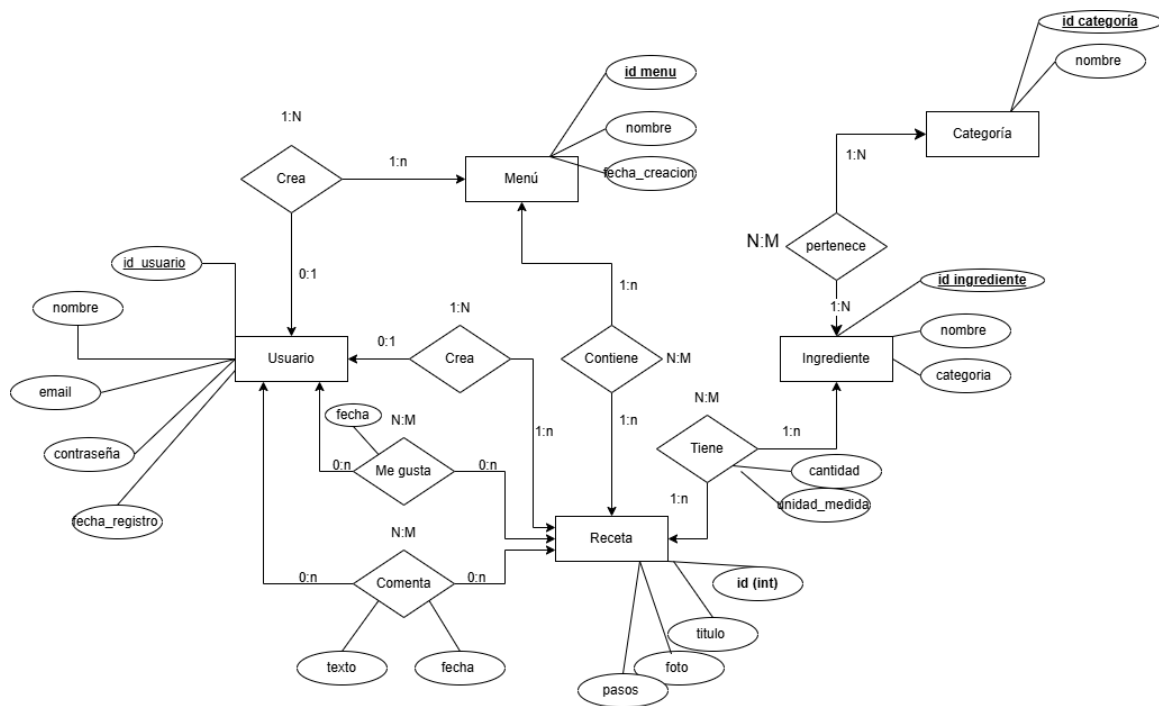


Imagen 10. Diagrama revisado de la base de datos durante la planificación inicial.

Desarrollo del backend

El 12 de enero se implementó el sistema de autenticación. Inicialmente se probó la autenticación CSRF de Laravel Breeze, pero se descartó por incompatibilidad con React. Se optó por Bearer Token con Laravel Sanctum, el mecanismo recomendado para APIs consumidas por aplicaciones SPA.

```
c:\laragon\www\Foodsynk>php artisan breeze:install

Which Breeze stack would you like to install?
Blade with Alpine ..... blade
Livewire (Volt Class API) with Alpine ..... livewire
Livewire (Volt Functional API) with Alpine ..... livewire-functional
React with Inertia ..... react
Vue with Inertia ..... vue
API only ..... api
api
api

Which testing framework do you prefer? [Pest]
Pest ..... 0
PHPUnit ..... 1
0
0
```

Imagen 11. Selección del modo API durante la instalación de Laravel Breeze.

El 13 de enero se instaló Laravel Orion para la generación automática del CRUD de recetas, configurado con Sanctum. También se implementó la RecipePolicy para que cada usuario solo pueda modificar sus propias recetas.

```
C:\laragon\www\Foodsyнк(main -> origin)
λ php artisan route:list

GET|HEAD / .....
GET|HEAD api/user .....
POST email/verification-notification .. verification.send > Auth\EmailVerificationNotificationController@store
POST forgot-password ..... password.email > Auth\PasswordResetLinkController@store
POST login ..... login > Auth\AuthenticatedSessionController@store
POST logout ..... logout > Auth\AuthenticatedSessionController@destroy
POST register ..... register > Auth\RegisteredUserController@store
POST reset-password ..... password.store > Auth\NewPasswordController@store
GET|HEAD sanctum/csrf-cookie ..... sanctum.csrf-cookie > Laravel\Sanctum > CsrfCookieController@show
GET|HEAD storage/{path} ..... storage.local
GET|HEAD up .....
GET|HEAD verify-email/{id}/{hash} ..... verification.verify > Auth\VerifyEmailController

Showing [12] routes
```

Imagen 12. Migración del token Sanctum generada al instalar Breeze en modo API.

Desarrollo del frontend

El 15 de enero se inició el frontend con React y Vite. Se configuró el AuthContext para gestionar el token de forma centralizada y el interceptor de Axios para incluirlo automáticamente. También se resolvió el error de CORS configurando cors.php y actualizando el .env del backend.

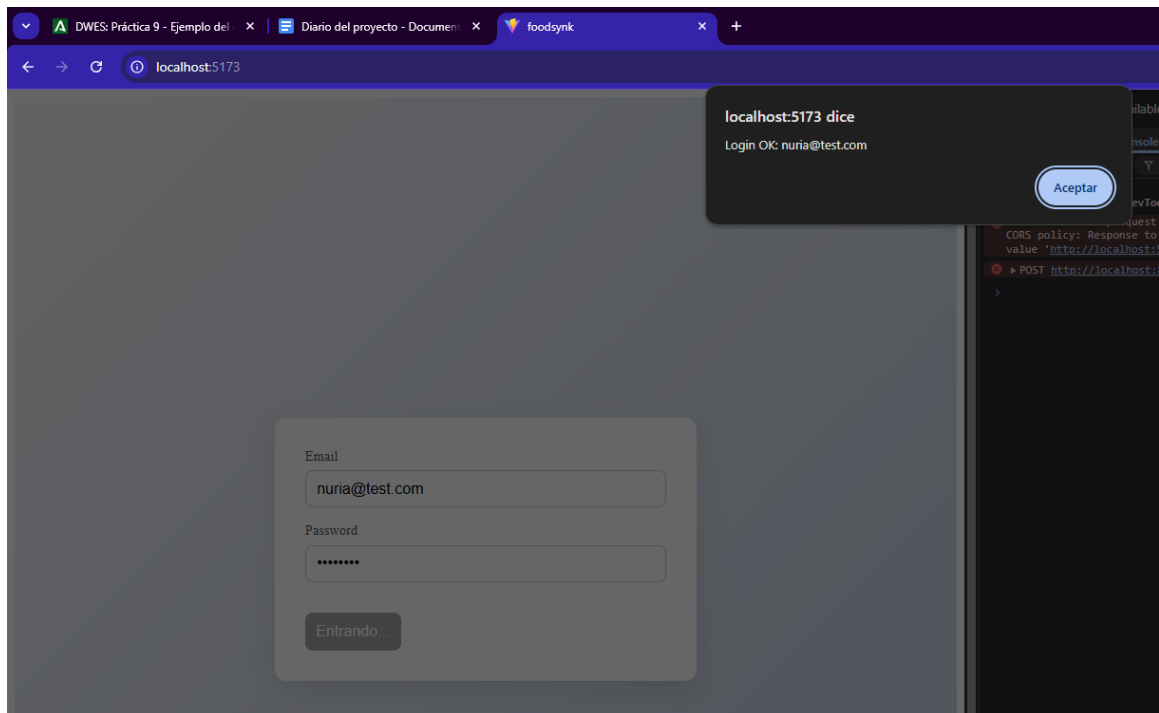


Imagen 13. Configuración de CORS resuelta — el backend acepta peticiones del frontend.

Hardening de seguridad

El 9 de febrero se implementó el hardening planificado en Antigravity: rate limiting en login y registro (OWASP A07), tabla de auditoría `user_access_logs`, validación con `FormRequests` (OWASP A03) y control de acceso con `RecipePolicy` (OWASP A01). Todo el trabajo se realizó en una rama Git separada (`Security`).

Plan de Implementación - Configuración de Limite de Velocidad (Rate Limiting)

Este plan describe los pasos para configurar el límite de velocidad (throttling) para los endpoints de la API en la aplicación Laravel.

Revisión del Usuario Requerida

NOTE

Configuraré un límite predeterminado de **60 peticiones por minuto** para la API. Esto se puede ajustar en  `App\Providers\AppServiceProvider.php`.

Cambios Propuestos

Configuración

[MODIFICAR]  `AppServiceProvider.php`

- Agregar el facade `RateLimiter`.
- Definir un limitador global o específico en el método  `boot`.
- Configuración de ejemplo: Limitar a 60 peticiones por minuto por dirección IP.

Imagen 14. Plan de implementación del rate limiting generado con apoyo de IA (Antigravity).

```
7 use App\Http\Controllers\Api\AuthTokenController;  
8 use App\Http\Controllers\Api\RegisteredUserController;  
9  
10 Route::middleware(middleware: ['throttle:5,1'])->post(uri: '/login', action: [AuthTokenController::c  
11 Route::middleware(middleware: ['throttle:5,1'])->post(uri: '/register', action: [RegisteredUserContr  
12  
13
```

Imagen 15. Rutas de login y registro con middleware throttle:5,1 aplicado.

Completar el CRUD de recetas

El 21 de febrero se reestructuró la base de datos añadiendo la tabla categories, la columna time en recetas y las tablas pivot ingredient_recipe y category_recipe. El 23 de febrero se completaron los seeders (100+ categorías, 26 ingredientes, 10 recetas) y se realizó el CRUD completo en Postman.

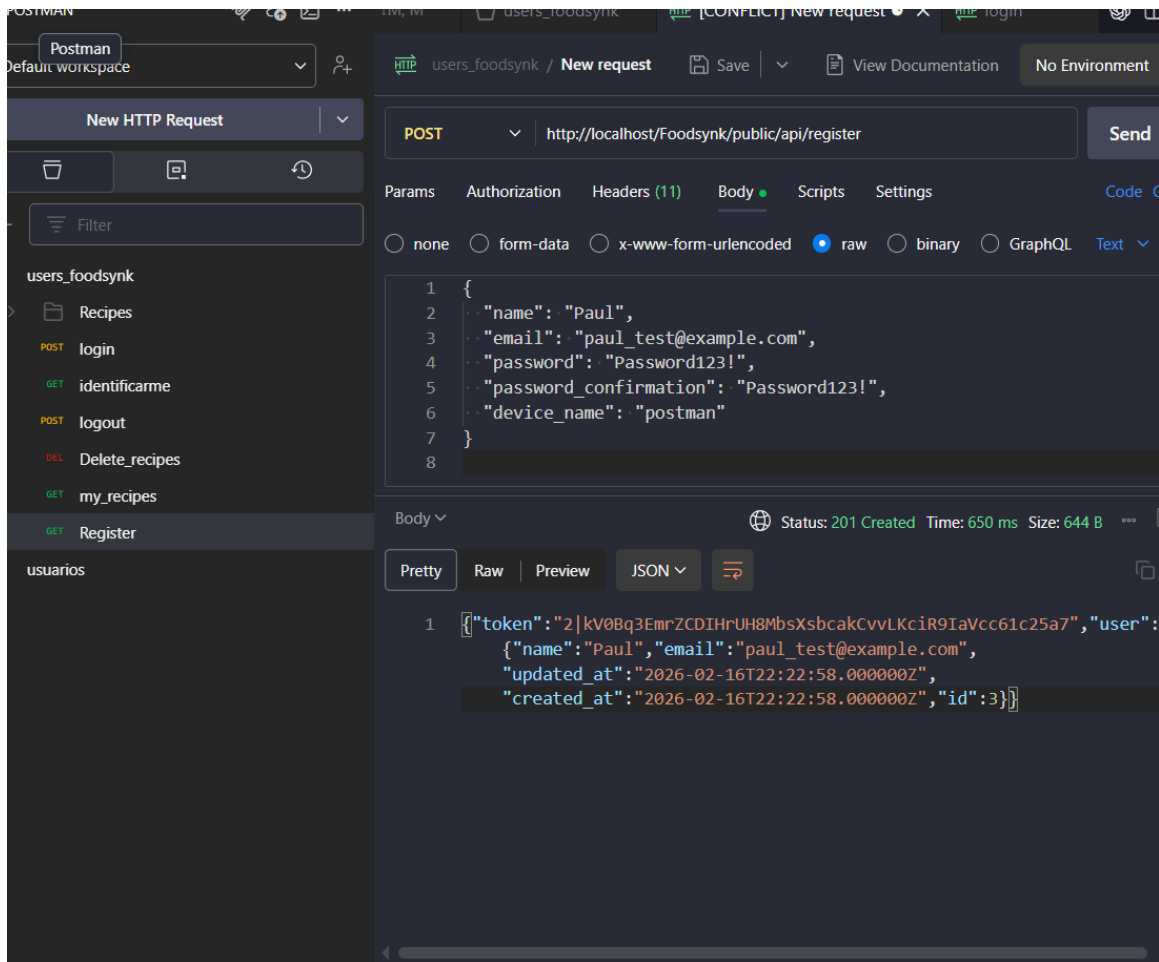


Imagen 16. Pruebas completas en Postman: registro, login, creación de recetas y rate limiting.

```

C:\laragon\www\Foodsynk(main -> origin)
λ php artisan tinker
Psy Shell v0.12.18 (PHP 8.3.30 - cli) by Justin Hileman
New PHP manual is available (latest: 3.0.1). Update with `doc --update-manual`
> \App\Models\UserAccessLog::latest()->take(20)->get();
= Illuminate\Database\Eloquent\Collection {#6543
  all: [
    App\Models\UserAccessLog {#6522
      id: 8,
      user_id: 3,
      action: "login",
      success: 0,
      ip: "127.0.0.1",
      user_agent: "PostmanRuntime/7.39.1",
      created_at: "2026-02-16 22:45:12",
      updated_at: "2026-02-16 22:45:12",
    },
    App\Models\UserAccessLog {#6521
      id: 7,
      user_id: 3,
      action: "login",
      success: 0,
      ip: "127.0.0.1",
      user_agent: "PostmanRuntime/7.39.1",
      created_at: "2026-02-16 22:45:11",
      updated_at: "2026-02-16 22:45:11",
    },
    App\Models\UserAccessLog {#6520
      id: 6,
      user_id: 3,
      action: "login",
      success: 0,
      ip: "127.0.0.1",
      user_agent: "PostmanRuntime/7.39.1",
      created_at: "2026-02-16 22:45:10",
      updated_at: "2026-02-16 22:45:10",
    },
    App\Models\UserAccessLog {#6519

```

middleware.

Nos vamos a la terminal

Vamos a probar que funcio

Imagen 17. Tabla user_access_logs con registros reales de login y logout verificados.

Tecnologías utilizadas

Frontend

- React: librería JavaScript para construir interfaces de usuario mediante componentes reutilizables. Permite actualizar solo las partes necesarias de la pantalla sin recargar toda la página.
- Vite: entorno de desarrollo para React con compilación rápida y recarga en caliente.
- Tailwind CSS: framework de estilos utilidad para construir interfaces modernas directamente en el HTML.
- Axios: cliente HTTP para JavaScript configurado con un interceptor que añade el Bearer Token automáticamente.
- React Router DOM: biblioteca para gestionar la navegación entre páginas dentro de una SPA sin recargar el navegador.

Backend

- Laravel 11: framework PHP completo, con estructura organizada, validación, sistema de rutas y ORM para construir APIs REST.
- Laravel Sanctum: paquete de autenticación para generar y validar Bearer Tokens de sesión.
- Laravel Breeze (modo API): herramienta de scaffolding instalada en modo API para trabajar con React como frontend separado.
- Laravel Orion: paquete que genera automáticamente los endpoints CRUD con paginación y filtros a partir del modelo Eloquent.
- PHP: lenguaje de programación del backend sobre el que se ejecuta Laravel.

Base de datos

- MySQL: sistema de gestión de bases de datos relacional para almacenar usuarios, recetas, ingredientes, categorías y menús.
- Eloquent ORM: sistema de mapeo objeto-relacional de Laravel que parametriza las consultas automáticamente, previniendo inyecciones SQL.

Infraestructura y herramientas

- Laragon: entorno de servidor local para Windows con Apache, PHP y MySQL.
- Git + GitHub: control de versiones con gestión por ramas (main, Security, etc.).
- Postman: herramienta para probar los endpoints de la API y verificar las medidas de seguridad.
- VSCode: editor de código utilizado para el desarrollo del proyecto.

Seguridad de la aplicación

La seguridad de FoodSynk se ha planificado desde el inicio tomando como referencia el OWASP Top 10. El plan de trabajo se documentó en el archivo Antigravity y las medidas se implementaron de forma incremental.

```
C:\laragon\www\FoodSynk(main -> origin)
λ php artisan migrate:fresh --seed

Dropping all tables ..... 74.16ms DONE
[INFO] Preparing database.

Creating migration table ..... 12.23ms DONE
[INFO] Running migrations.

0001_01_01_000000_create_users_table ..... 53.84ms DONE
0001_01_01_000001_create_cache_table ..... 14.77ms DONE
0001_01_01_000002_create_jobs_table ..... 52.74ms DONE
2026_01_10_195858_create_ingredients_table ..... 15.72ms DONE
2026_01_10_203303_create_personal_access_tokens_table ..... 29.78ms DONE
2026_01_13_174610_create_recipes_table ..... 7.11ms DONE
2026_01_13_182521_add_user_id_to_recipes_table ..... 34.09ms DONE

[INFO] Seeding database.

Database\Seeders\UserRecipeSeeder ..... RUNNING
Database\Seeders\UserRecipeSeeder ..... 1,054 ms DONE

C:\laragon\www\FoodSynk(main -> origin)
```

Imagen 18. Base de datos con usuarios y recetas de prueba para verificar el funcionamiento.

Autenticación segura (OWASP A07)

La autenticación se implementó con Bearer Token mediante Laravel Sanctum. Al cerrar sesión, el token se elimina del servidor impidiendo su reutilización. Se configuró rate limiting a 5 intentos por minuto por IP: si se supera, el servidor devuelve un error 429 (Too Many Requests). Los mensajes de error no revelan si el correo existe en el sistema, evitando la enumeración de usuarios. En el frontend, el token se almacena en sessionStorage (no en localStorage) para que se elimine al cerrar la pestaña.

Control de acceso (OWASP A01)

El control de acceso se implementa con Laravel Policies. La RecipePolicy garantiza que solo el propietario puede editar o eliminar sus recetas: cualquier otro usuario recibe un 403 Forbidden. Este control está en el backend (siempre) y en el frontend (ocultando los botones al que no es propietario). Para fases futuras está planificado un middleware de roles para rutas de administración y una MenuPolicy que evite el acceso IDOR a menús ajenos.

Validación de datos (OWASP A03)

La validación de datos de entrada se centraliza en clases FormRequest (StoreRecipeRequest, UpdateRecipeRequest, RecipeRequest) con reglas estrictas por campo. Un campo inválido devuelve automáticamente un error 422. El uso de Eloquent

ORM parametriza las consultas, previniendo inyecciones SQL. En el frontend, Axios escapa los datos antes de enviarlos.

Auditoría de accesos

La tabla `user_access_logs` registra todos los intentos de login, logout y registro, guardando el usuario, tipo de acción, si fue exitosa, IP (hasta 45 caracteres para IPv6), user agent y marca de tiempo. Permite detectar intentos de acceso no autorizado.

Diseño seguro (OWASP A04)

Para las fases futuras está planificado validar límites en operaciones complejas (evitar menús con número excesivo de días que saturen el servidor) y verificar la existencia de los recursos referenciados antes de procesarlos.

Público objetivo

FoodSynk está pensada para cualquier persona que quiera organizar sus comidas de forma más eficiente, sin conocimientos avanzados de cocina ni de tecnología:

- Personas que compran para una familia y necesitan planificar los menús de la semana para hacer una sola compra completa.
- Estudiantes o personas que viven solas y quieren evitar improvisar qué comer cada día.
- Cualquier persona interesada en llevar una alimentación más variada y equilibrada.

La aplicación no requiere conocimientos específicos. Su interfaz está pensada para ser clara e intuitiva, de forma que cualquier usuario pueda empezar a usarla sin instrucciones previas.

Mejoras futuras

FoodSynk está diseñada de forma modular, lo que facilita añadir nuevas funcionalidades sin rehacer la arquitectura existente.

Funcionalidades pendientes (versión 2)

- Módulo de menús semanales: permite crear un menú semanal seleccionando recetas por día y comida, indicando el número de personas.
- Lista de la compra automática: genera los ingredientes necesarios con cantidades ajustadas al número de personas, con opción de editar, imprimir y descargar.
- Sistema de favoritos y comentarios: los usuarios podrán marcar recetas como favoritas y comentar recetas de otros.
- Roles de administrador: panel de gestión para moderar recetas, comentarios y usuarios, con rutas protegidas por middleware de roles (OWASP A01).
- Despliegue en producción: frontend en Vercel y backend en VPS con Nginx, SSL (Let's Encrypt) y copias de seguridad automatizadas.

Mejoras técnicas

- Subida real de imágenes: actualmente solo se admite URL. En la versión 2 se implementará la subida de archivos al servidor.
- Panel de administrador visual unificado para gestionar usuarios, recetas y comentarios.
- Tests automatizados con PHPUnit: login fallido 5 veces, intento de borrar receta ajena, validaciones de entrada.
- Verificación de correo electrónico tras el registro.
- Mejoras de accesibilidad: contraste de colores, etiquetas ARIA y navegación por teclado.

Escalabilidad futura

- Cálculo de valores nutricionales a partir de los ingredientes de cada receta.
- Sistema de recomendaciones según los menús y recetas más usadas por el usuario.
- Agrupación de ingredientes por categoría en la lista de la compra para facilitar el recorrido por el supermercado.